

Data 236- Distributed Systems

Homework-1

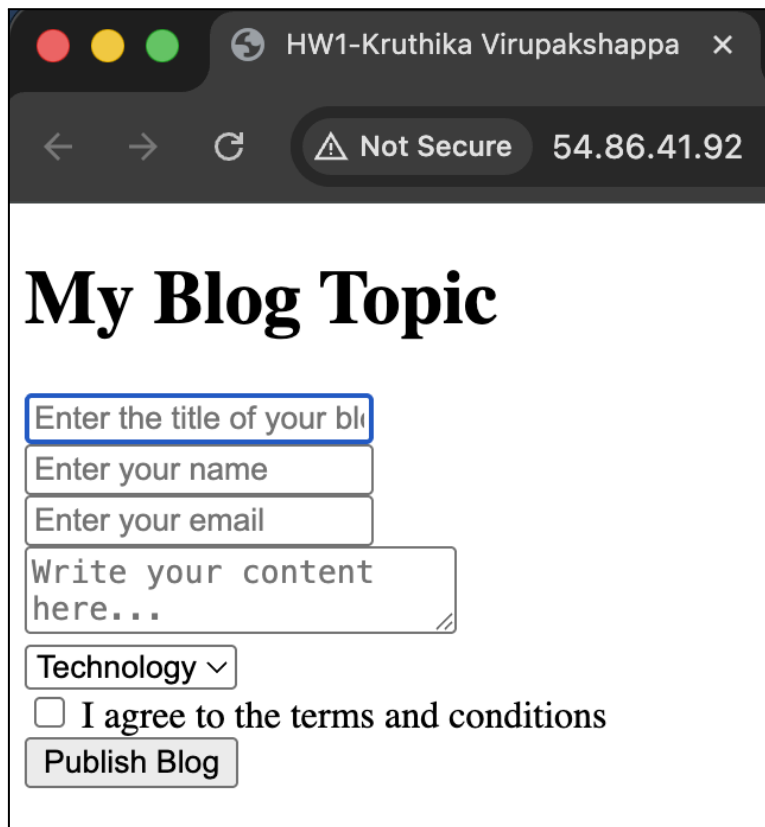
Kruthika Virupakshappa

Part 1

HTML

1. Create an HTML page with the title "HW1-{Your Name}".

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>HW1-Kruthika Virupakshappa</title>
  </head>
```

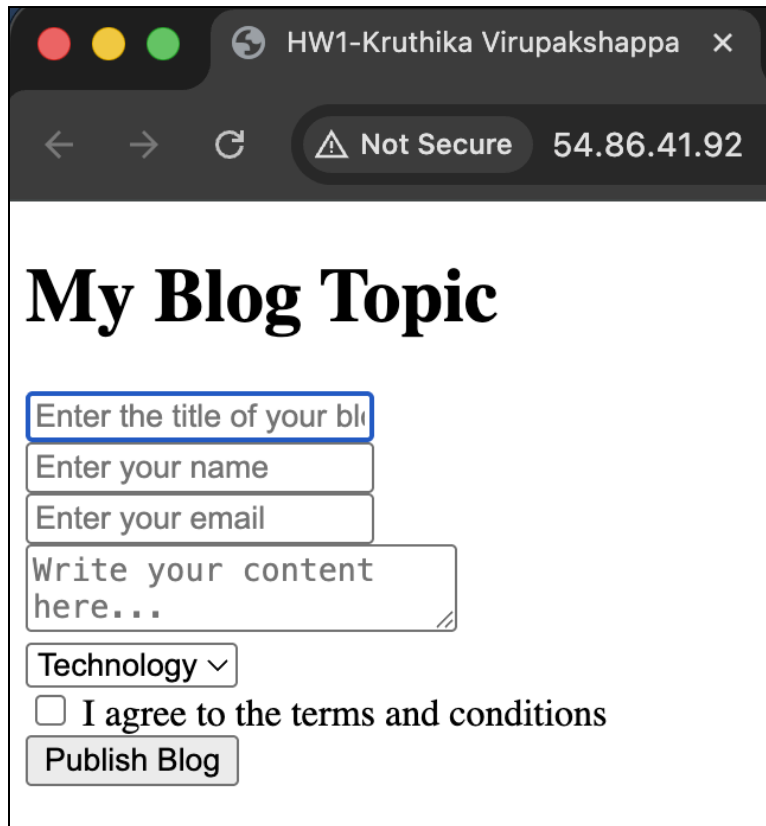


A screenshot of a web browser window. The title bar shows 'HW1-Kruthika Virupakshappa'. The address bar shows 'Not Secure' and the IP address '54.86.41.92'. The main content area has a heading 'My Blog Topic'. Below the heading is a form with the following elements: a text input field with placeholder text 'Enter the title of your blog', a text input field with placeholder text 'Enter your name', a text input field with placeholder text 'Enter your email', a text area with placeholder text 'Write your content here...', a dropdown menu with 'Technology' selected, a checkbox labeled 'I agree to the terms and conditions', and a 'Publish Blog' button.

2. Add a heading tag to name the title of the blog topic. (Note. you have to use the header tag which renders the biggest font size).

H1 stands for heading 1 which is the biggest font size

```
<h1> My Blog Topic</h1>
```



The screenshot shows a web browser window with the title 'HW1-Kruthika Virupakshappa'. The address bar shows '54.86.41.92' and a 'Not Secure' warning. The main content area displays a form titled 'My Blog Topic' in a large, bold, black font. The form includes several input fields: a text input for the blog title with placeholder text 'Enter the title of your bl', a text input for the author's name with placeholder text 'Enter your name', a text input for the email address with placeholder text 'Enter your email', and a text area for the blog content with placeholder text 'Write your content here...'. Below these fields is a dropdown menu for category selection, currently showing 'Technology'. There is also a checkbox for 'I agree to the terms and conditions' and a 'Publish Blog' button.

3. Create a form for the blog. The form should include the following:
 - a. A text input for the blog title, placeholder text "Enter the title of your blog". This field should be required and the cursor should automatically focus on this field when the page loads.
 - b. A text input for the author name, placeholder "Enter your name", and required.
 - c. An email input for the email address, placeholder "Enter your email", and required.
 - d. A text area for the blog content, placeholder "Write your content here...", and required.
 - e. A dropdown for category selection, and options "Technology," "Lifestyle," "Travel," and "Education."
 - f. A checkbox and a label with the text "I agree to the terms and conditions."
 - g. A submit button with the text "Publish Blog".

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>HW1-Kruthika Virupakshappa</title>
  </head>
  <body>
    <h1> My Blog Topic</h1>
    <form>
      <input id="title" type="text" name="title" placeholder="Enter the
title of your blog" required autofocus/>
      <br>
      <input id="name" type="text" name="name" placeholder="Enter your
name" required />
      <br>
      <input id="email" type="email" name="email" placeholder="Enter your
email" required />
      <br>
      <textarea id="content" name="content" placeholder="Write your
content here..." required></textarea>
      <br>

      <select id="category" name="category" required>
        <option value="Technology">Technology</option>
        <option value="Lifestyle">Lifestyle</option>
        <option value="Travel">Travel</option>
        <option value="Education">Education
        </option>
      </select>
      <br>

      <input type="checkbox" id="agree">
      <label for="agree">I agree to the terms and conditions</label>
      <br>
      <input type="submit" value="Publish Blog"/>
    </form>

    <script src="script.js"> </script>
  </body>
</html>
```

HW1-Kruthika Virupakshappa

Not Secure 54.86.41.92

My Blog Topic

Enter the title of your blog

Enter your name

Enter your email

Write your content here...

Technology ▾

☐ I agree to the terms and conditions

Publish Blog

4. Add a script tag to link your javascript code for part II at the end of your HTML file.

```
<script src="script.js"> </script>
```

Javascript:

1. Write a javascript function using an arrow function to validate:
 - a. Verify if the blog content is more than 25 characters. If the validation fails, display an alert with the message "Blog content should be more than 25 characters".
 - b. Verify if the terms and conditions check box is checked. If the validation fails, display an alert with the message "You must agree to the terms and conditions".

```
const validateForm = () => {  
  const contentValue = document.getElementById("content").value.trim();  
  const agreeChecked = document.getElementById("agree").checked;  
  if (contentValue.length <= 25) {  
    alert("Blog content should be more than 25 characters");  
    return false;  
  }  
}
```

```

    if (!agreeChecked) {
        alert("You must agree to the terms and conditions");
        return false;
    }

    return true;
};

```

To test blog with less than 25 character error:

To test blog with terms and condition error:

2. After the form submission is successful, convert the form data into a JSON string and log the output in the console.

```

const formData = new FormData(form);
const dataObj = Object.fromEntries(formData.entries());

const jsonString = JSON.stringify(dataObj);
console.log("JSON String:", jsonString);

```

After submission, JavaScript prevents page reload, validates input, converts form data to JSON, logs processed data to the console and tracks successful submissions

3. Use object destructuring to extract the title and email fields from the parsed object and log their values in the console.

```

const {title, email} = parsedObj;

```

```
console.log("Title:", title);  
console.log("Email:", email);
```

Object destructuring is used to extract the title and email properties from the parsed JSON object and log them to the console.

4. Use the spread operator to add a new field “submissionDate” with the current date and time to the parsed object. Log the updated parsed object in the console.

```
const updatedObj = {  
  ...parsedObj,  
  submissionDate: new Date().toISOString()  
};  
console.log("Updated Object:", updatedObj);
```

...parsedObj is the spread operator. It copies all the existing properties from parsedObj into a new object. Then a new field submissionDate is added.

5. Create a closure to track how many times the form has been successfully submitted and log the submission count each time the form is submitted.

```
const createSubmissionCounter = () => {  
  let count = 0;  
  return () => {  
    count += 1;  
    return count;  
  };  
};
```

In this part, count is defined inside createSubmissionCounter. The inner arrow function remembers count even after createSubmissionCounter finishes

```
const incrementSubmissionCount = createSubmissionCounter();
```

This runs createSubmissionCounter() once. IncrementSubmissionCount holds the inner function. Same count variable is reused every time.

```
const submissionCount = incrementSubmissionCount();  
console.log("Submission Count:", submissionCount);
```

Use the closure on each successful submit

JSON String: {"title":"test title","name":"test name","email":"test@email.com","content":"asasasasasasasdcnknebcuwebcueb","category":"Technology"} [script.js:38](#)

Title: test title [script.js:43](#)

Email: test@email.com [script.js:44](#)

Updated Object: [script.js:50](#)

▶ {title: 'test title', name: 'test name', email: 'test@email.com', content: 'asasasasasasasdcnknebcuwebcueb', category: 'Technology', ...}

Submission Count: 1 [script.js:53](#)

JSON String: {"title":"test title","name":"test name","email":"test@email.com","content":"asasasasasasasdcnknebcuwebcueb","category":"Technology"} [script.js:38](#)

Title: test title [script.js:43](#)

Email: test@email.com [script.js:44](#)

Updated Object: [script.js:50](#)

▶ {title: 'test title', name: 'test name', email: 'test@email.com', content: 'asasasasasasasdcnknebcuwebcueb', category: 'Technology', ...}

Submission Count: 2 [script.js:53](#)

JSON String: {"title":"test title","name":"test name","email":"test@email.com","content":"asasasasasasasdcnknebcuwebcueb","category":"Technology"} [script.js:38](#)

Title: test title [script.js:43](#)

Email: test@email.com [script.js:44](#)

Updated Object: [script.js:50](#)

▶ {title: 'test title', name: 'test name', email: 'test@email.com', content: 'asasasasasasasdcnknebcuwebcueb', category: 'Technology', ...}

Submission Count: 3 [script.js:53](#)

> `cmd` `i` to turn on code suggestions. [Don't show again](#) NEW

JavaScript:

```
const createSubmissionCounter = () => {
  let count = 0;
  return () => {
    count += 1;
    return count;
  };
};

const incrementSubmissionCount = createSubmissionCounter();

const validateForm = () => {
```

```
const contentValue = document.getElementById("content").value.trim();
const agreeChecked = document.getElementById("agree").checked;
    if (contentValue.length <= 25) {
        alert("Blog content should be more than 25 characters");
        return false;
    }

    if (!agreeChecked) {
        alert("You must agree to the terms and conditions");
        return false;
    }

    return true;
};

const form = document.querySelector("form");
form.addEventListener("submit", (event) => {
    event.preventDefault();

    if (!validateForm()) return;

    const formData = new FormData(form);
    const dataObj = Object.fromEntries(formData.entries());

    const jsonString = JSON.stringify(dataObj);
    console.log("JSON String:", jsonString);

    const parsedObj = JSON.parse(jsonString);

    const {title, email} = parsedObj;
    console.log("Title:", title);
    console.log("Email:", email);

    const updatedObj = {
        ...parsedObj,
        submissionDate: new Date().toISOString()
    };
    console.log("Updated Object:", updatedObj);

    const submissionCount = incrementSubmissionCount();
    console.log("Submission Count:", submissionCount);
```

```
    alert("Form submitted successfully!");  
});
```

Deployment:

Docker: Create a docker image of the above application, and build and run the application using docker in your local.

Provide the screenshot of your app running on your localhost.

```
my_blog_app — docker run --name blog-container -p 8080:80 blog-app — 116x44  
  
=> => exporting layers 0.0s  
=> => exporting manifest sha256:9cda96af87a4c6535e04f67ec6a41cb359f0e359 0.0s  
=> => exporting config sha256:2fa809bb2f3a8827dc4902a41298bd3ad93a4d13f9 0.0s  
=> => exporting attestation manifest sha256:ef303165cf26acc54c0a3af647a1 0.0s  
=> => exporting manifest list sha256:22350c50f11565090cbdd4f731397deb9ab 0.0s  
=> => naming to docker.io/library/blog-app:latest 0.0s  
=> => unpacking to docker.io/library/blog-app:latest 0.0s  
  
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cvhiikh4jwbbrgdadw2akucsp  
(env) Kruthika@Kruthikas-MacBook-Air my_blog_app % docker run --name blog-container -p 8080:80 blog-app  
  
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration  
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh  
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf  
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf  
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh  
/docker-entrypoint.sh: Configuration complete; ready for start up  
2026/02/01 21:57:30 [notice] 1#1: using the "epoll" event method  
2026/02/01 21:57:30 [notice] 1#1: nginx/1.29.4  
2026/02/01 21:57:30 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)  
2026/02/01 21:57:30 [notice] 1#1: OS: Linux 6.10.14-linuxkit  
2026/02/01 21:57:30 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576  
2026/02/01 21:57:30 [notice] 1#1: start worker processes  
2026/02/01 21:57:30 [notice] 1#1: start worker process 30  
2026/02/01 21:57:30 [notice] 1#1: start worker process 31  
2026/02/01 21:57:30 [notice] 1#1: start worker process 32  
2026/02/01 21:57:30 [notice] 1#1: start worker process 33  
2026/02/01 21:57:30 [notice] 1#1: start worker process 34  
2026/02/01 21:57:30 [notice] 1#1: start worker process 35  
2026/02/01 21:57:30 [notice] 1#1: start worker process 36  
2026/02/01 21:57:30 [notice] 1#1: start worker process 37  
192.168.65.1 - - [01/Feb/2026:21:57:44 +0000] "GET / HTTP/1.1" 200 1290 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-"  
192.168.65.1 - - [01/Feb/2026:21:57:44 +0000] "GET /script.js HTTP/1.1" 200 1380 "http://localhost:8080/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-"  
2026/02/01 21:57:44 [error] 31#31: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 192.168.65.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "localhost:8080", referer: "http://localhost:8080/"  
192.168.65.1 - - [01/Feb/2026:21:57:44 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:8080/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-"
```

Containers [Give feedback](#)

Container CPU usage ?

3.70% / 800% (8 CPUs available)

Container memory usage ?

1.43GB / 7.47GB

Show charts

☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU...	Memory usage...	Actions
☐	● blog-container	e65a327aff74	blog-app	8080:80	0%	7.39MB / 7.65GB	

localhost:8080

My Blog Topic

Write your content here...

Technology

☐ I agree to the terms and conditions

AWS ECS: Create an AWS ECS service (only one task) running the above application using the docker image created above.

Provide the screenshot of your app running on your public IP address.

← → ↻ ⚠ Not Secure 54.86.41.92

My Blog Topic

Enter the title of your blog

Enter your name

Enter your email

Write your content here...

Technology ▾

☐ I agree to the terms and conditions

Publish Blog

aws

Search

[Option+S] Ask Amazon Q

United States (N. Virginia)

Krutika Vinayakshappa (171038cb16964060b9cb6b9c328d6e51)

Krutika Vinayakshappa

Amazon Elastic Container Service

Clusters > blog-app-cluster > Services > blog-app-service > Tasks > 171038cb16964060b9cb6b9c328d6e51 > Configuration

Configuration Metrics Logs Networking Volumes (0) Tags

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR ^L

Repositories ^L

AWS Batch ^L

Online learning workshop ^L ^{New}

Documentation ^L

Discover products ^L

Task overview

ARN
arn:aws:ecs:us-east-1:376142074491:task/blog-app-cluster/171038cb16964060b9cb6b9c328d6e51

Last status
Running

Desired status
Running

Started/Created at
February 1, 2026, 20:07 (UTC-8:00)
February 1, 2026, 20:06 (UTC-8:00)

Fargate ephemeral storage

Encryption ^{Info}
Default AWS Fargate encryption

Size (GiB)
20

Configuration

Operating system/Architecture
Linux/X86_64

CPU | Memory
.25 vCPU | 5 GiB

Capacity provider
-

Launch type
Fargate

ENI ID
eni-Defa1c9cb35947ed4 ^L

Network mode
awsvpc

Public IP
54.86.41.92 | [open address](#) ^L

Private IP
172.31.93.34

Part-2 Agentic AI

Build two tiny “agents” that talk to each other using a small local LLM (via **Ollama**).
Input: a blog **title** and **content**. Output: exactly **3 topical tags** + a **≤25-word summary**
— produced through a short Planner → Reviewer → Finalizer flow — and printed as **valid JSON**.

Questions:

1. **Set up** a small local LLM with Ollama.(use smollm:1.7b)
2. **Write and Run** a Python script that creates two agents (Planner, Reviewer) and a finalization step.
3. **Capture** outputs and submit a PDF with your console screenshots + answers.

```
import json
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage

schema = {
    "type": "object",
    "properties": {
        "tags": {
            "type": "array",
            "items": {"type": "string"},
            "minItems": 3,
            "maxItems": 3
        },
        "summary": {"type": "string", "minLength": 80, "maxLength": 200}
    },
    "required": ["tags", "summary"],
    "additionalProperties": False
}

def call_ollama():
    try:
        llm = ChatOllama(
            model="smollm:1.7b",
            temperature=0.7,
            format=schema,
            num_predict=256
        )
        return llm
    except Exception as e:
        return f"Error: {str(e)}"

#Agent 1: Planner
def planner_agent(title, content):
    """
    Planner Agent: Takes the input of blog's title and it's content to give the
    desired response.
    """
    prompt = f"""You are a SUMMARIZATION EXPERT. Analyze the following text and provide
    exactly:
    1. Three relevant tags (1-3 words each)
    2. A concise summary of 20-25 words that captures the main idea
```

Strict Rules:

Generate a NEW summary in your own words. Do not copy sentences from the text.
Focus on the core concept and purpose.

TITLE: {title}

TEXT: {content}

Return ONLY valid JSON with "tags" array and "summary" string."""

```
llm = call_ollama()
resp = llm.invoke([HumanMessage(content=prompt)])
data = json.loads(resp.content)
```

```
tags = data["tags"]
summary = data["summary"]
return tags, summary
```

```
def reviewer_agent(title, content, tags, summary):
```

```
    prompt = f"""You are a review agent.
```

Your task:

- Validate the provided tags and summary
- Correct them ONLY if they violate the rules
- If they are already correct, return them unchanged

Rules:

- tags: exactly 3 items, each 1-3 words, relevant to the text
- summary: exactly 1 sentence, maximum 25 words, abstractive
- Do NOT invent placeholder values
- Do NOT return example text
- Use the actual content to decide

Return ONLY valid JSON with keys:

- tags
- summary

TITLE:

{title}

TEXT:

{content}

CURRENT TAGS:

{tags}

CURRENT SUMMARY:

{summary}

"""

```
llm = call_ollama()
resp = llm.invoke([HumanMessage(content=prompt)])
data = json.loads(resp.content)
```

```

tags = data["tags"]
summary = data["summary"]
return tags, summary

def finalization_step(tags, summary):
    """
    Finalization Step: validation and strict JSON output.
    """

    try:
        tags = tags[:3]
        while len(tags) < 3:
            tags.append("fill")

        if not isinstance(summary, str):
            summary = ""
        summary = summary.strip()
        words = summary.split()
        if len(words) > 25:
            summary = " ".join(words[:25]).rstrip(".") + "."
        if not summary:
            raise ValueError("Could not find valid summary")

        return json.dumps(
            {"tags": tags, "summary": summary},
        )

    except Exception as e:
        return f"Error: {str(e)}"

def main():

    title = "Build and run docker Image"
    content = """
        Docker Image is a read-only template to build containers. An image holds all
        the information needed to bootstrap a container, including what processes
        to run and the configuration data. Every image starts from a base image,
        and a template is created by using the instructions that are stored in the
        DockerFile. For each instruction, a new layer is created on the image.
        •Running the container originates from the image we created in the previous
        step. When a container is launched, a read-write layer is added to the top of
        the
        image. After appropriate network and IP address allocation, the desired
        application can now be run inside the container."""

    print("="*60)
    print(f"Title: {title}")
    print(f"Content: {content}")
    print()

    #Planner Agent
    print("="*60)

```

```

print("Output from the planner agent:\n")
planner_result = planner_agent(title, content)
print(f"TAGS: {planner_result[0]}\n")
print(f"SUMMARY: {planner_result[1]}\n")
print()

# Reviewer Agent
print("="*60)
print("Output from the Reviewer agent:\n")
reviewer_result = reviewer_agent(title, content, planner_result[0], planner_result[1])
print(f"TAGS: {reviewer_result[0]}\n")
print(f"SUMMARY: {reviewer_result[1]}\n")

# #Finalization Agent
print("="*60)
print("Result from the Finalizer step:\n")
final_result = finalization_step(reviewer_result[0], reviewer_result[1])
print(final_result)
print()

if __name__ == "__main__":
    main()

```

Deliverables:

1. Command Used
python agents_demo.py
2. Console screenshots

Planner output

```

(env) Kruthika@Kruthikas-MacBook-Air HW1 % python agents_demo.py
=====
Title: Build and run docker Image
Content: Docker Image is a read-only template to build containers. An image holds all
the information needed to bootstrap a container, including what processes
to run and the configuration data. Every image starts from a base image,
and a template is created by using the instructions that are stored in the
DockerFile. For each instruction, a new layer is created on the image.
•Running the container originates from the image we created in the previous
step. When a container is launched, a read-write layer is added to the top of the
image. After appropriate network and IP address allocation, the desired
application can now be run inside the container.
=====
Output from the planner agent:
TAGS: ['docker', 'image', 'build']
SUMMARY: Build and run a new docker image from a base image, starting from a template and adding instructions to the application.
=====

```

Reviewer output

```
=====
Output from the Reviewer agent:
TAGS: ['docker', 'image', 'build']
SUMMARY: Build and run a new docker image from a base image, starting from a template and adding instructions to the application.
=====
```

Finalized output

```
=====
Result from the Finalizer step:
{"tags": ["docker", "image", "build"], "summary": "Build and run a new docker image from a base image, starting from a template and adding instructions to the application."}
=====
```

3. Short answers

Q1. Write the 3 final tags you obtained.

Answer: ["docker", "image", "build"]

Q2. Paste the final summary (≤25 words).

Answer: Build and run a new docker image from a base image, starting from a template and adding instructions to the application.

Q3. Did the Reviewer agent change anything? (yes/no + explanation)

Answer: No

Explanation: The reviewer was giving strict guidelines to keep it unchanged if the planner output is satisfying the initial conditions, the reason to do this was to avoid any uncertainty since the temperature was set 0.7(High) for creative results. Below are some of the guidelines for the reviewer.

- Validate the provided tags and summary
- Correct them ONLY if they violate the rules
- If they are already correct, return them unchanged

4. **Explanation** of the each step in your own words

1. Imports:

```
import json
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage
```

json, ChatOllama and HumanMessage (used to send prompts to an Ollama model through LangChain).

2. schema

```
schema = {
    "type": "object",
    "properties": {
        "tags": {
            "type": "array",
            "items": {"type": "string"},
            "minItems": 3,
            "maxItems": 3
        },
        "summary": {"type": "string", "minLength": 80, "maxLength": 200}
    },
    "required": ["tags", "summary"],
    "additionalProperties": False
}
```

Here we define the expected JSON format the model must return. It must return an object with only tags and summary. Tags must have exactly 3 items and summary must be between 80 to 200 characters

3. call_ollama()

```
def call_ollama():
    try:
        llm = ChatOllama(
            model="smollm:1.7b",
            temperature=0.7,
            format=schema,
            num_predict=256
        )
        return llm
    except Exception as e:
        return f"Error: {str(e)}"
```

This function initializes the Ollama model with settings

4. Planner agent:

```
def planner_agent(title, content):
    """
    Planner Agent: Takes the input of blog's title and it's content to give the
    desired response.
    """
    prompt = f"""You are a SUMMARIZATION EXPERT. Analyze the following text and
    provide exactly:
```

```

1. Three relevant tags (1-3 words each)
2. A concise summary of 20-25 words that captures the main idea

Strict Rules:
Generate a NEW summary in your own words. Do not copy sentences from the
text.

Focus on the core concept and purpose.

TITLE: {title}
TEXT: {content}

Return ONLY valid JSON with "tags" array and "summary" string."""
llm = call_ollama()
resp = llm.invoke([HumanMessage(content=prompt)])
data = json.loads(resp.content)

tags = data["tags"]
summary = data["summary"]
return tags, summary

```

The planner agent creates the first draft and then it calls the model and parses JSON. It sends a prompt to the model and the model returns JSON and extracts tags and summary.

5. Reviewer Agent:

```

def reviewer_agent(title, content, tags, summary):
    prompt = f"""You are a review agent.

    Your task:
    - Validate the provided tags and summary
    - Correct them ONLY if they violate the rules
    - If they are already correct, return them unchanged

    Rules:
    - tags: exactly 3 items, each 1-3 words, relevant to the text
    - summary: exactly 1 sentence, maximum 25 words, abstractive
    - Do NOT invent placeholder values
    - Do NOT return example text
    - Use the actual content to decide

    Return ONLY valid JSON with keys:
    - tags
    - summary

```

```

TITLE:
{title}

TEXT:
{content}

CURRENT TAGS:
{tags}

CURRENT SUMMARY:
{summary}
"""

llm = call_ollama()
resp = llm.invoke([HumanMessage(content=prompt)])
data = json.loads(resp.content)

tags = data["tags"]
summary = data["summary"]
return tags, summary

```

This checks the planner output and fixes it only if needed. It calls the model again. It uses the content to check relevance and rules. It returns corrected tags and summary if rules are violated.

6. Finalization step:

```

def finalization_step(tags, summary):
    """
    Finalization Step: validation and strict JSON output.
    """

    try:
        tags = tags[:3]
        while len(tags) < 3:
            tags.append("fill")

        if not isinstance(summary, str):
            summary = ""
        summary = summary.strip()
        words = summary.split()
        if len(words) > 25:
            summary = " ".join(words[:25]).rstrip(".") + "."
        if not summary:

```

```

        raise ValueError("Could not find valid summary")

    return json.dumps(
        {"tags": tags, "summary": summary},
    )
except Exception as e:
    return f"Error: {str(e)}"

```

This is the finalization step to guarantee a proper result. It ensures the summary is a string and not empty. It cuts extra tags. It trims summary to max 25 words. It returns JSON String.

7. main(): It gives input, runs all the agents and prints the output.

```

def main():

    title = "Build and run docker Image"
    content = """Docker Image is a read-only template to build containers. An image holds all
        the information needed to bootstrap a container, including what processes
        to run and the configuration data. Every image starts from a base image,
        and a template is created by using the instructions that are stored in the
        DockerFile. For each instruction, a new layer is created on the image.
        •Running the container originates from the image we created in the previous
        step. When a container is launched, a read-write layer is added to the top of
        the
        image. After appropriate network and IP address allocation, the desired
        application can now be run inside the container."""

    print("="*60)
    print(f"Title: {title}")
    print(f"Content: {content}")
    print()

    #Planner Agent
    print("="*60)
    print("Output from the planner agent:\n")
    planner_result = planner_agent(title, content)
    print(f"TAGS: {planner_result[0]}\n")
    print(f"SUMMARY: {planner_result[1]}\n")
    print()

    # Reviewer Agent
    print("="*60)
    print("Output from the Reviewer agent:\n")
    reviewer_result = reviewer_agent(title, content, planner_result[0], planner_result[1])
    print(f"TAGS: {reviewer_result[0]}\n")
    print(f"SUMMARY: {reviewer_result[1]}\n")

    # #Finalization Agent
    print("="*60)
    print("Result from the Finalizer step:\n")

```

```
final_result = finalization_step(reviewers_result[0], reviewers_result[1])
print(final_result)
print()

if __name__ == "__main__":
    main()
```